Atividade Avaliativa 2

Sistema de Agendamento de Monitorias

Luiz Bonfá – João Lourenço

Repositório: `https://github.com/lwyo1/AT2---ENG-DE-SOFTWARE`

1 Projeto Inicial

História escolhida (prioridade Alta): 1. Marcar horário
Como aluno, eu quero selecionar uma das disciplinas e marcar um horário de monitoria para tirar minhas dúvidas sem necessidade de entrar em contato por mensagem.

Classes e responsabilidades

Todo o código fica na pasta `codigo/`, em Python, sem interface gráfica. Os dados ficam em memória (listas dentro dos objetos), sem banco de dados.

- **Materia** (`materia.py`): guarda o nome da disciplina, a lista de tópicos do conteúdo a partir de um leitor de CSV com esses dados e os horários cadastrados pelo monitor. Oferece `horarios_livres()`, que devolve só os horários da disciplina que não estão cancelados nem lotados.
- **Horario** (`horario.py`): representa um atendimento de 30 minutos de um monitor em uma disciplina. Guarda data e hora de início, tipo (individual ou grupo), capacidade (1 no individual, definida pelo monitor no grupo), se está cancelado e as reservas feitas nele. Responde se ainda tem vaga (`tem_vaga()`) e permite `cancelar()`.
- **Reserva** (`reserva.py`): liga um estudante a um horário e ao tópico escolhido.
- **Estudante** (`usuario.py`, classe abstrata): guarda RA, nome e as reservas da pessoa. Conta quantas reservas ela tem em uma disciplina numa semana. Define o método abstrato `pode_reservar(horario)`.
- **Aluno** (filha de Estudante): `pode_reservar` aceita qualquer disciplina.
- **Monitor** (filha de Estudante): guarda a disciplina que monitora e cadastra seus horários. Como também é aluno, pode reservar horários de outras disciplinas, mas `pode_reservar` recusa horários da própria disciplina.
- **Agendamento** (`agendamento.py`): função `marcar_horario(estudante, horario, topico)`, que verifica todas as regras na ordem e cria a reserva. Quando alguma regra falha, lança a exceção `ReservaInvalida` com o motivo. O limite semanal fica na constante `LIMITE_SEMANAL = 3`, fácil de mudar caso o cliente decida outro valor.

Testes de aceitação

Arquivo `codigo/test_marcar_horario.py`, com `unittest`. Cada teste monta seus próprios objetos, sem depender de entrada digitada. Comando: `python -m unittest discover codigo`.

Critério de aceitação	**Testes**	**Cenário**
1. Só aparecem horários livres da disciplina escolhida	`test_lista_apenas_horarios_livres_da_disciplina` `test_horario_lotado_nao_aparece`	sucesso / falha
2. Horário individual reservado não aceita outra reserva	`test_individual_aceita_primeira_reserva` `test_individual_recusa_segunda_reserva`	sucesso / falha
3. Horário de grupo aceita reservas até a capacidade	`test_grupo_aceita_ate_capacidade` `test_grupo_recusa_acima_da_capacidade`	sucesso / falha

4. No máximo 3 reservas por semana na mesma disciplina	`test_terceira_reserva_na_semana_aceita`
	`test_quarta_reserva_na_semana_recusada`
	`test_limite_ignora_outra_semana_e_outra_disciplina`
		sucesso / falha / sucesso

5. A reserva exige um tópico do conteúdo	`test_reserva_com_topico_valido`
	`test_reserva_sem_topico_recusada`
	`test_topico_de_outra_disciplina_recusado`
		sucesso / falha / falha

6. Horário cancelado não pode ser reservado	`test_horario_cancelado_nao_aparece` `test_horario_cancelado_recusa_reserva`	falha / falha

Também teremos `test_monitor_reserva_outra_disciplina` e `test_monitor_nao_reserva_propria_disciplina`, para a regra do Monitor como estudante.

2 Relatório de Execução

Como executar

- **Python:** 3.14
- **Bibliotecas:** nenhuma externa, apenas a biblioteca padrão.
- **Arquivo principal:** `codigo/main.py`.
  - Na pasta raiz: `python codigo/main.py`
  - Dentro de `codigo/`: `python main.py`
- **Testes:**
  - Na pasta raiz: `python -m unittest discover codigo`
  - Dentro de `codigo/`: `python -m unittest -v`
- **Logins de exemplo:** aluno `2669552` / `iwhbhg`; monitor de Cálculo 1 `2655120` / `mn42rk`.
- No menu, `0` troca de usuário; a opção "Fechar o programa" encerra (4 para aluno, 5 para monitor).
- Para voltar aos horários de demonstração, apague `codigo/horarios.csv` e `codigo/reservas.csv`.

Plano e execução

Etapa	Responsável	Planejado	Realizado
Histórias de usuário	Luiz	até 17/10	19/10, Luiz
Projeto inicial	João	até 18/10	19/10, commit do Luiz devido a imprevisto
Implementação e testes	Luiz e João	até 21/10	20 e 21/10. Luiz: classes do domínio, `agendamento.py`, 18 testes. João: login, menu, horários de exemplo, cadastro, persistência
Documentação	Luiz e João	22/10	22/10
Autoavaliação	Luiz e João	22/10	22/10

Atrasos e mudanças em relação ao projeto inicial

- A princípio, não foi possível realizar nenhuma etapa nos dois primeiros dias (17 e 18/10) por situações inesperadas que ocorreram no fim de semana. No dia 19, foram feitas as etapas pendentes referentes às histórias de usuário e ao projeto inicial.
- Foram feitos outros recursos no código que não estavam no projeto inicial, como o menu, a tela da agenda, a persistência e o login, que facilitam o funcionamento e a demonstração do sistema.
- O sistema de persistência de dados em arquivo CSV não foi planejado. A princípio, o sistema realizaria todos os processos em memória, e os dados seriam perdidos ao fechar o programa. Para trazer mais consistência, decidimos salvar essas alterações em arquivo.
- O cadastro de horários não ficou na classe `Monitor`; ele virou uma opção do menu, apenas para que o sistema possa ser usado. O cadastro completo fica para a história 2.
- Foi adicionado um terceiro teste no critério 1, `test_horario_de_outra_disciplina_nao_aparece`, que verifica que um horário livre de outra disciplina não aparece na lista da disciplina escolhida. Os testes previstos no projeto inicial não cobriam a parte "da disciplina escolhida" do critério.
- O teste `test_limite_ignora_outra_semana_e_outra_disciplina` (critério 4) foi dividido em `test_limite_ignora_outra_semana` e `test_limite_ignora_outra_disciplina`, para que, se um falhar, fique claro qual regra está com erro.
- No planejamento, seria feito um sistema de notificação e confirmação para o monitor assim que um aluno realizasse um agendamento. Porém, para economizar tempo e simplificar a implementação inicial, o agendamento é realizado sem a necessidade de confirmação.
- A primeira implementação aborda a história 1 com alguns recursos de outras histórias; por isso, o usuário Professor e o sistema de relatórios ficaram de fora.
- No e-mail, é sugerido um limite de 2 reservas semanais por disciplina, porém decidimos permitir 3, já que o e-mail não exigia esse número e deixou a escolha em aberto. O valor fica na constante `LIMITE_SEMANAL`, fácil de alterar.

3 Documentação Técnica

3.1 Histórias de Usuário

1. Marcar horário

Como **aluno**, eu quero selecionar uma das disciplinas e marcar um horário de monitoria para tirar minhas dúvidas sem necessidade de entrar em contato por mensagem.

Critérios de aceitação:

- Só aparecem horários livres da disciplina escolhida.
- Em horário individual, depois de reservado ele não aceita outra reserva.
- Em horário de grupo, aceita reservas até a capacidade definida pelo monitor.
- O aluno pode ter no máximo 3 reservas por semana na mesma disciplina.
- A reserva exige a seleção de um dos tópicos do conteúdo.
- Horário cancelado não pode ser reservado.

Prioridade: Alta
Justificativa: É o problema central, a superlotação de horários.
Trecho do e-mail: "um monitor de Cálculo tinha três alunos marcados no mesmo horário"

2. Cadastrar disponibilidade

Como **monitor**, eu quero cadastrar meus horários de atendimento para que os alunos possam reservá-los.

Critérios de aceitação:

- Cada horário tem duração de 30 minutos.
- O monitor define se o horário é individual ou em grupo e, se for em grupo, a capacidade.
- Não é possível cadastrar dois horários sobrepostos para o mesmo monitor.

Prioridade: Alta
Justificativa: Sem horários cadastrados, a reserva não existe.
Trecho do e-mail: "cada monitor informa os horários em que pode atender"

3. Cancelar horário e avisar alunos

Como **monitor**, eu quero cancelar um horário para que os alunos saibam antes de ir até a sala.

Critérios de aceitação:

- O horário cancelado deixa de aparecer como livre.
- Todos os alunos com reserva naquele horário recebem um aviso.
- As reservas do horário cancelado não contam no limite semanal do aluno.

Prioridade: Média
Justificativa: Resolve uma reclamação direta dos alunos.
Trecho do e-mail: "só descobrem o cancelamento quando chegam na sala"

4. Relatório mensal de horas

Como **coordenação**, eu quero gerar um relatório mensal com as horas atendidas por monitor para calcular a bolsa.

Critérios de aceitação:

- Só contam atendimentos em que o monitor registrou o comparecimento.
- Cada atendimento conta 30 minutos (atendimento em grupo conta uma vez só).
- Apenas usuários com perfil de coordenação acessam o relatório.

Prioridade: Média
Justificativa: Importante para o pagamento, mas depende da reserva funcionando.
Trecho do e-mail: "relatório mensal com as horas atendidas por cada monitor"

5. Ver tópicos com mais dúvidas

Como **professor**, eu quero ver quais tópicos da minha disciplina tiveram mais reservas para reforçar esses conteúdos em aula.

Critérios de aceitação:

- O professor vê apenas as disciplinas dele.
- Os tópicos aparecem ordenados pela quantidade de reservas.
- O relatório não mostra dados de bolsa nem de horas dos monitores.

Prioridade: Baixa
Justificativa: Traz benefício, mas não é urgente para o funcionamento.
Trecho do e-mail: "gostariam de saber quais conteúdos geram mais dúvidas"

3.2 Testes de Aceitação

História implementada: 1. Marcar horário. Arquivo `codigo/test_marcar_horario.py`, 18 testes. Cada classe de teste corresponde a um critério de aceitação, mais uma classe para a regra do monitor, e todas herdam de `CenarioBase`, que cria uma matéria, um monitor e um aluno novos antes de cada teste. Os testes chamam `marcar_horario` diretamente, sem entrada digitada, e nos cenários em que a reserva é recusada e que nenhuma reserva foi registrada.

Critério de aceitação	Teste	Cenário
1. Só aparecem horários livres da disciplina escolhida	`test_lista_apenas_horarios_livres_da_disciplina`	sucesso
	`test_horario_lotado_nao_aparece`	falha
	`test_horario_de_outra_disciplina_nao_aparece`	falha
2. Horário individual reservado não aceita outra reserva	`test_individual_aceita_primeira_reserva`	sucesso
	`test_individual_recusa_segunda_reserva`	falha
3. Horário de grupo aceita reservas até a capacidade	`test_grupo_aceita_ate_capacidade`	sucesso
	`test_grupo_recusa_acima_da_capacidade`	falha
4. No máximo 3 reservas por semana na mesma disciplina	`test_terceira_reserva_na_semana_aceita`	sucesso
	`test_quarta_reserva_na_semana_recusada`	falha
	`test_limite_ignora_outra_semana`	sucesso
	`test_limite_ignora_outra_disciplina`	sucesso
5. A reserva exige um tópico do conteúdo	`test_reserva_com_topico_valido`	sucesso
	`test_reserva_sem_topico_recusada`	falha
	`test_topico_de_outra_disciplina_recusado`	falha
6. Horário cancelado não pode ser reservado	`test_horario_cancelado_nao_aparece`	falha
	`test_horario_cancelado_recusa_reserva`	falha
Extra: monitor não reserva a própria disciplina	`test_monitor_reserva_outra_disciplina`	sucesso
	`test_monitor_nao_reserva_propria_disciplina`	falha

O que não foi coberto

- Login, menu, cadastro de horário e persistência não têm testes automatizados: dependem de entrada digitada ou de arquivos, e são extras fora da história 1.
- O mesmo aluno pode reservar duas vagas do mesmo horário em grupo, e a história 1 não trata disso.
- O critério 6 só tem cenários de falha, porque o caso de sucesso (reservar um horário não cancelado) já acontece em todos os testes de sucesso dos outros critérios.